

Lista Dao - slisXAUE

Security Assessment

CertiK Assessed on Jun 8th, 2026





Certik Assessed on Jun 8th, 2026

Lista Dao - slisXAUE

The security assessment was prepared by Certik.

Executive Summary

TYPES

DeFi

ECOSYSTEM

EVM Compatible

METHODS

Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 05/30/2026

Final report published on 06/09/2026

Vulnerability Summary

15

Total Findings

3

Resolved

0

Partially Resolved

12

Acknowledged

0

Declined

3 Centralization

3 Acknowledged



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

1 Medium

1 Acknowledged



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

5 Minor

1 Resolved, 4 Acknowledged



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

6 Informational

2 Resolved, 4 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | LISTA DAO - SLISXAUE

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Review Notes

[Overview](#)

[Privileged Functions](#)

[External Dependencies](#)

Findings

[LDS-01 : Centralized Control Of Contract Upgrade](#)

[LDS-02 : Centralization Related Risks](#)

[LDS-03 : Centralized Withdrawn Risk](#)

[LDS-04 : Excessive NAV Price Volatility Leads To Denial Of Service](#)

[LDS-05 : `deposit\(\)` Does Not Round Up The Amount](#)

[LDS-06 : Potential Bad Debt Issue When `SlisXAUE.totalSupply\(\)` Is Zero](#)

[LDS-07 : Withdrawal Requests May Be Blocked By The `minRedeemShares` Constraint](#)

[LDS-08 : Deposit And Request Withdraw Lack On-Chain Slippage Protection](#)

[LDS-16 : Pre-Sync On `setXAUEOracle` Blocks Oracle Migration When Current Oracle Causes Sync Revert](#)

[LDS-09 : Variable And Comment Inconsistency](#)

[LDS-10 : User-Facing Transfers Credit Nominal Amount Instead Of Received Balance](#)

[LDS-11 : `DecreaseTotalAssets` Event Emits Uncapped Input Amount](#)

[LDS-13 : Fee Collection During NAV Price Fluctuations](#)

[LDS-14 : Confirmation Of `depositToVault`, `requestWithdrawFromVault` And `acknowledgeReject` Delays In The Protocol](#)

[LDS-15 : Queued Withdrawal Amount Locking Leads To Loss Shifting Onto Remaining Shareholders](#)

Optimizations

[LDS-12 : Unbounded Withdrawal Request Arrays Lead to Degrading Bulk Read Scalability](#)

Appendix

Disclaimer

CODEBASE | LISTA DAO - SLISXAUE

Repository

<https://github.com/lista-dao/lista-new-contracts>

Commit

- [b5f3b9cb59c8c8c60150d30f21ba1f6db7f1080e](#)
- [92af28f8266de6bb22c2d7670fb906b63585a5e5](#)

AUDIT SCOPE | LISTA DAO - SLISXAUE

lista-dao/lista-new-contracts

 SlisXAUE.sol

 XAUEAdapter.sol

 XAUTStaking.sol

 XAUEAdapter.sol

 SlisXAUE.sol

 XAUTStaking.sol

APPROACH & METHODS | LISTA DAO - SLISXAUE

This audit was conducted for Lista Dao to evaluate the security and correctness of the smart contracts associated with the Lista Dao - slisXAUE project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | LISTA DAO - SLISXAUE

Overview

The **Lista slisXAUE** is a three-contract XAUT staking stack that wraps external XAUE Protocol yield into a transferable share token (`slisXAUE`). All three core contracts are UUPS-upgradeable and use OpenZeppelin-style

`AccessControlEnumerableUpgradeable` role gating.

`SlisXAUE.sol` is a ERC20 share token with no business logic and no pause. Only the `MINTER` role (granted to `XAUTStaking` at deployment) can mint and burn shares. Shares are freely transferable to preserve DeFi composability.

`XAUTStaking.sol` is the user-facing vault: users deposit `XAUT` (6-dec) to receive `slisXAUE` (18-dec) shares, `requestWithdraw` burns shares immediately and queues a FIFO batch withdrawal, and `claimWithdraw` transfers `XAUT` back once the adapter funds the batch. Share pricing is driven by `userTotalAssetsScaled` and updated only via adapter callbacks (`increaseTotalAssets` / `decreaseTotalAssets`). The contract supports configurable `mintCap` , `minDeposit` , `minWithdraw` , pausing (`PAUSER` / `MANAGER`), and `emergencyWithdraw` .

`XAUEAdapter.sol` bridges `XAUTStaking` to the external XAUE Protocol. It holds Lista's XAUE `FundToken` share balance, syncs NAV-based profit/loss via `updateVaultAssets()` , charges a protocol fee on upside, and pushes net interest or loss to staking. `BOT` moves `XAUT` into/out of XAUE (`depositToVault` , `requestWithdrawFromVault`), forwards redemption proceeds to staking (`finishEarnPoolWithdraw`), claims fees (`claimFee`), and handles rejected redemptions (`acknowledgeReject`). Accounting is based on `expectedShareBalance` rather than raw `balanceOf` . `MANAGER` can tune fee parameters, NAV delta caps, and invoke `emergencyWithdraw` .

Privileged Functions

In the **Lista slisXAUE** project, privileged roles are used to support runtime governance, upgradeability, parameter tuning, and emergency response, which are specified in the finding `Centralization Related Risks` , `Centralized Control of Contract Upgrade` and `Centralized Withdrawn Risk` .

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community.

It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should be also considered to move to the execution queue of the `TimeLock` contract.

External Dependencies

Third-party libraries used in scope:

- `@openzeppelin/contracts`

- @openzeppelin/contracts-upgradeable

Configurable or external addresses by contract:

- **SlisXAUE**: role-bearing admin accounts (DEFAULT_ADMIN_ROLE, MINTER).
- **XAUTStaking**: asset (XAUT), slisXAUE, adapter, and role-bearing admin accounts (DEFAULT_ADMIN_ROLE, MANAGER, PAUSER).
- **XAUEAdapter**: immutable asset (XAUT), staking, xaueFundToken, xaueOracle, feeReceiver, and role-bearing admin accounts (DEFAULT_ADMIN_ROLE, MANAGER, BOT).

External protocol dependencies (outside scope, assumed trusted):

- **XAUE FundToken** (CoboFundToken): sync mint(), async requestRedemption(), redemptions() getter, whitelist gate, and REDEMPTION_APPROVER settlement/rejection flow.
- **XAUE Oracle** (CoboFundOracle): getLatestPrice() NAV feed (1e18 scale).
- **XAUT**: standard 6-decimal ERC20 underlying asset.

It is assumed that these external contracts/addresses and privileged operators are trusted and correctly managed across the **Lista slisXAUE** deployment lifecycle. In particular, XAUEAdapter must be whitelisted on the XAUE FundToken before any mint/redeem can succeed, and the adapter/staking addresses must be correctly cross-wired at initialization.

FINDINGS | LISTA DAO - SLISXAUE



15

Total Findings

0

Critical

3

Centralization

0

Major

1

Medium

5

Minor

6

Informational

This report has been prepared for Lista Dao to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 15 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
LDS-01	Centralized Control Of Contract Upgrade	Centralization	Centralization	● Acknowledged
LDS-02	Centralization Related Risks	Centralization	Centralization	● Acknowledged
LDS-03	Centralized Withdrawn Risk	Centralization	Centralization	● Acknowledged
LDS-04	Excessive NAV Price Volatility Leads To Denial Of Service	Coding Issue	Medium	● Acknowledged
LDS-05	<code>deposit()</code> Does Not Round Up The Amount	Logical Issue	Minor	● Resolved
LDS-06	Potential Bad Debt Issue When <code>SlisXAUE.totalSupply()</code> Is Zero	Logical Issue	Minor	● Acknowledged
LDS-07	Withdrawal Requests May Be Blocked By The <code>minRedeemShares</code> Constraint	Logical Issue	Minor	● Acknowledged
LDS-08	Deposit And Request Withdraw Lack On-Chain Slippage Protection	Logical Issue, Design Issue	Minor	● Acknowledged
LDS-16	Pre-Sync On <code>setXAUEOracle</code> Blocks Oracle Migration When Current Oracle Causes Sync Revert	Logical Issue	Minor	● Acknowledged
LDS-09	Variable And Comment Inconsistency	Inconsistency	Informational	● Resolved

ID	Title	Category	Severity	Status
LDS-10	User-Facing Transfers Credit Nominal Amount Instead Of Received Balance	Volatile Code	Informational	● Acknowledged
LDS-11	<code>DecreaseTotalAssets</code> Event Emits Uncapped Input Amount	Logical Issue	Informational	● Resolved
LDS-13	Fee Collection During NAV Price Fluctuations	Design Issue	Informational	● Acknowledged
LDS-14	Confirmation Of <code>depositToVault</code> , <code>requestWithdrawFromVault</code> And <code>acknowledgeReject</code> Delays In The Protocol	Design Issue	Informational	● Acknowledged
LDS-15	Queued Withdrawal Amount Locking Leads To Loss Shifting Onto Remaining Shareholders	Coding Issue, Design Issue	Informational	● Acknowledged

LDS-01 | Centralized Control Of Contract Upgrade

Category	Severity	Location	Status
Centralization	● Centralization	SlisXAUE.sol (05/27-b5f3b9): 13; XAUEAdapter.sol (05/27-b5f3b9): 42; XAUTStaking.sol (05/27-b5f3b9): 27~32	● Acknowledged

Description

In the contract `XAUTStaking`, `XAUEAdapter` and `SlisXAUE`, the role `DEFAULT_ADMIN_ROLE` has the authority to update the implementation contract behind the proxy contract.

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (2/3, 3/5) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (2/3, 3/5) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.

- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

I Alleviation

[Lista Dao, 06/03/2026]:

Acknowledged. Admin will be TimeLock contract.

[CertiK, 06/04/2026]:

It is suggested to implement the aforementioned methods to avoid centralized failure. CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

We will review the transactions once contracts have been deployed and update the finding status accordingly.

LDS-02 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	● Centralization	SlisXAUE.sol: 49, 55, 60; XAUEAdapter.sol: 159, 181, 205, 219, 255, 261, 267, 275, 308, 408; XAUTStaking.sol: 266, 284, 302, 357, 363, 369, 375, 381, 385, 389	● Acknowledged

Description

In the contract `SlisXAUE`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

Inherited from `AccessControlUpgradeable`:

- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

Inherited from `UUPSUpgradeable`:

- `upgradeToAndCall(address newImplementation, bytes memory data)`

The `DEFAULT_ADMIN_ROLE` is initially granted to the `admin` address passed to `initialize(address admin, address minter, string memory name_, string memory symbol_)`. Because no custom role admin is configured, this role is the admin for itself and for `MINTER`; therefore, a holder can add or remove administrators and minters through the inherited role-management functions. A compromised or malicious holder could grant token minting and burning authority to an arbitrary account, revoke the intended minter, or upgrade the UUPS proxy to new logic. The upgrade authority materially expands the role's effective control: even if the current implementation only exposes limited token controls, the administrator may deploy new logic with broader behavior, so current implementation-level restrictions may not be binding in practice. The main limitation is that `upgradeToAndCall` is only reachable through the proxy context and the constructor disables initializers on the implementation contract; these limitations do not reduce the proxy administrator's ongoing authority over token logic, role administration, and token trust assumptions.

In the contract `SlisXAUE`, the role `MINTER` has authority over the following functions:

Defined in `SlisXAUE`:

- `mint(address to, uint256 amount)`
- `burn(address from, uint256 amount)`

The `MINTER` role is initially granted to the `minter` address passed to `initialize`, and the comments indicate this is intended to be `XAUTstaking`. The `DEFAULT_ADMIN_ROLE` can later grant or revoke this role through `grantRole` and `revokeRole`. A compromised or malicious `MINTER` holder could mint arbitrary `slisXAUE` shares to any address, diluting existing holders and affecting share-based accounting, or burn tokens from any address without requiring user approval, disrupting user balances and redemption rights. The role does not itself have authority to grant roles, revoke roles, or upgrade the contract unless separately granted administrative privileges. Its impact is limited to token supply changes, but

those supply changes directly affect user balances, fairness, redemption accounting, and the economic integrity of the staking product.

In the contract `XAUEAdapter`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

Inherited from `AccessControlUpgradeable`:

- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

Inherited from `UUPSUpgradeable`:

- `upgradeToAndCall(address newImplementation, bytes memory data)`

The `DEFAULT_ADMIN_ROLE` is initially granted to the `_admin` address passed to `initialize`. Because no custom role admin is configured, this role administers itself, `MANAGER`, and `BOT`. A compromised or malicious holder could grant or revoke operational roles, remove legitimate operators, assign role authority to arbitrary accounts, or upgrade the adapter proxy to new logic. The upgrade authority is especially sensitive because the adapter holds XAUE share balances, accounts for NAV changes, routes withdrawal funds to staking, charges fees, and interacts with the XAUE oracle and fund token. Even if the current implementation restricts manager and bot operations with caps, validation checks, and nonzero-address requirements, the administrator may gain broader effective control by upgrading to new logic, so the current restrictions may not be binding in practice. The main limitation is that the inherited upgrade function is proxy-only and role-gated, but a valid administrator remains a central trust assumption for funds, accounting, and protocol operation.

In the contract `XAUEAdapter`, the role `MANAGER` has authority over the following functions:

Defined in `XAUEAdapter`:

- `setFeeReceiver(address _feeReceiver)`
- `setFeeRate(uint256 _feeRate)`
- `setMaxDeltaBps(uint256 _maxDeltaBps)`
- `setXAUEOracle(address _xaueOracle)`
- `setStaking(address _staking)`
- `emergencyWithdraw(address token, uint256 amount)`

The `MANAGER` role is initially granted to the `_manager` address passed to `initialize`, and it can be reassigned by `DEFAULT_ADMIN_ROLE`. A compromised or malicious manager could redirect protocol fees by changing `feeReceiver`, change the fee rate applied to future NAV gains, adjust the NAV delta cap that gates accounting updates, replace the XAUE oracle used for vault valuation, repoint the staking contract that receives interest and withdrawal callbacks, or withdraw arbitrary ERC20 tokens held by the adapter to the manager address through `emergencyWithdraw`. These privileges can affect fee distribution, valuation, accounting, withdrawal routing, and the custody of assets or XAUE shares held by the adapter. Meaningful implementation limits exist: `feeRate` is capped by `MAX_FEE_RATE` at 30%, `maxDeltaBps` must be greater than zero and no more than `MAX_DELTA_BPS_CAP` at 3000 basis points, configured addresses must be nonzero, and lowering `maxDeltaBps` settles accrued NAV under the existing cap first. These limits reduce but do not eliminate the trust assumption, particularly because `emergencyWithdraw` is not restricted to non-core tokens and because `setXAUEOracle` and `setStaking` can materially affect system accounting and liveness.

In the contract `XAUEAdapter`, the role `BOT` has authority over the following functions:

Defined in `XAUEAdapter`:

- `depositToVault(uint256 assetAmount)`
- `requestWithdrawFromVault(uint256 assetAmount)`
- `finishEarnPoolWithdraw(uint256 amount)`
- `claimFee(uint256 feeAmount)`
- `acknowledgeReject(uint256 reqId)`

The `BOT` role is initially granted to the `_bot` address passed to `initialize`, and it can be reassigned by `DEFAULT_ADMIN_ROLE`. A compromised or malicious bot could control the timing and amount of XAUT moved from the adapter into the XAUE vault, initiate XAUE redemption requests from the adapter's share balance, trigger delivery of adapter-held XAUT to `XAUTStaking` for withdrawal batch processing, claim accumulated fees to the configured fee receiver, or acknowledge rejected XAUE redemption requests and update the adapter's accounting baseline. These operations can affect withdrawal liquidity, vault exposure, fee collection timing, NAV synchronization, and operational liveness. The role has meaningful constraints: `depositToVault` requires a positive amount and XAUE minimum deposit compliance, `requestWithdrawFromVault` relies on current NAV and XAUE minimum redemption checks, `claimFee` cannot exceed the accumulated fee or the adapter's XAUT balance and pays only the configured `feeReceiver`, `acknowledgeReject` validates request ownership, rejected status, one-time acknowledgement, and sufficient returned shares, and all role-gated bot functions are non-reentrant. However, the role remains a centralized operational dependency for vault deposits, redemption requests, fee realization, rejected-redemption accounting, and withdrawal flow continuity.

In the contract `XAUTStaking`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

Inherited from `AccessControlUpgradeable`:

- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

Inherited from `UUPSUpgradeable`:

- `upgradeToAndCall(address newImplementation, bytes memory data)`

The `DEFAULT_ADMIN_ROLE` is initially granted to the `_admin` address passed to `initialize`. Because no custom role admin is configured, this role administers itself, `MANAGER`, and `PAUSER`. A compromised or malicious administrator could assign or remove managers and pausers, revoke legitimate operators, add new administrators, or upgrade the staking proxy to new logic. The upgrade authority materially expands the administrator's effective control over deposits, withdrawals, share accounting, pausing behavior, and asset custody: even if the current implementation appears limited, a privileged administrator may deploy new logic with broader controls, so current implementation-level restrictions may not be binding in practice. The main limitation is that the inherited upgrade function is only usable through the proxy and requires the role check in `_authorizeUpgrade`, but the administrator remains a central trust assumption for the user-facing staking system.

In the contract `XAUTStaking`, the role `MANAGER` has authority over the following functions:

Defined in `XAUTStaking`:

- `setMintCap(uint256 _mintCap)`

- `setMinDeposit(uint256 _minDeposit)`
- `setMinWithdraw(uint256 _minWithdraw)`
- `emergencyWithdraw(address token, uint256 amount)`
- `unpause()`

The `MANAGER` role is initially granted to the `_manager` address passed to `initialize`, and it can be reassigned by `DEFAULT_ADMIN_ROLE`. A compromised or malicious manager could raise, lower, or effectively disable the deposit capacity by changing `mintCap`, set deposit or withdrawal minimums to values that restrict user access, withdraw arbitrary ERC20 tokens held by the staking contract to the manager address through `emergencyWithdraw`, or resume the protocol after a pause. These powers can affect user access, withdrawal availability, operational fairness, and asset custody, especially because the staking contract may hold XAUT allocated for finalized withdrawal claims. Meaningful limitations exist: the manager cannot pause the system unless also granted `PAUSER`, cannot upgrade the contract or administer roles unless also granted `DEFAULT_ADMIN_ROLE`, and the current implementation does not expose manager setters for `asset`, `slisXAUE`, or `adapter`. The `unpause` function can only succeed while the contract is paused, and the parameter setters reject no-op updates, but these checks do not remove the centralized control over user limits and emergency asset movement.

In the contract `XAUTStaking`, the role `PAUSER` has authority over the following functions:

Defined in `XAUTStaking`:

- `pause()`

The `PAUSER` role is initially granted to the `_pauser` address passed to `initialize`, and it can be reassigned by `DEFAULT_ADMIN_ROLE`. A compromised or malicious pauser could pause the staking contract, which blocks `deposit`, `requestWithdraw`, and `claimWithdraw` because those user-facing functions are protected by `whenNotPaused`. This can affect protocol liveness and prevent users from entering, requesting exits, or claiming finalized withdrawals until a manager unpauses. The role is limited because it cannot unpause, withdraw assets, change parameters, grant roles, or upgrade the contract unless it also holds other roles. Adapter callbacks such as `increaseTotalAssets` and `finishWithdraw` are not protected by `whenNotPaused`, so the pause primarily affects user-facing actions rather than all internal accounting.

In the contract `XAUTStaking`, the role `adapter` has authority over the following functions:

Defined in `XAUTStaking`:

- `increaseTotalAssets(uint256 amount)`
- `finishWithdraw(uint256 amount)`

The `adapter` privilege is assigned during `initialize` through the `_adapter` address and is enforced with explicit `require(msg.sender == adapter)` checks. The current `XAUTStaking` implementation does not expose a public setter for `adapter`, so this is primarily an architecture-bound module role. However, the in-scope `XAUEAdapter` is itself upgradeable and controlled by its own `DEFAULT_ADMIN_ROLE`, `MANAGER`, and `BOT` roles, so compromise or abusive control of the adapter path can still affect staking behavior. A malicious or compromised adapter could call `increaseTotalAssets` to alter `userTotalAssetsScaled` and affect the share-to-asset conversion rate when there is nonzero `slisXAUE` supply, or call `finishWithdraw` to deliver XAUT and advance withdrawal batch confirmations. The role is limited because `finishWithdraw` transfers XAUT from the adapter when `amount > 0`, `amount = 0` can only consume existing

`withdrawQuota`, and the adapter role cannot directly call manager functions, pause the contract, upgrade the staking implementation, or emergency-withdraw assets. Nevertheless, users rely on the configured adapter and its administrators for accurate interest accounting, withdrawal funding, and withdrawal-batch liveness.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

■ Alleviation

[Lista Dao, 06/03/2026]: Acknowledged.

`DEFAULT_ADMIN_ROLE` -> TimeLock + Gnosis Safe (per LDS-01).

`MANAGER` / `PAUSER` -> ops Gnosis Safe multisig.

`BOT` -> restricted hot-key EOA (semi-trusted; only on-chain-validated actions).

[CertiK, 06/04/2026]:

It is suggested to implement the aforementioned methods to avoid centralized failure. CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

We will review the transactions once contracts have been deployed and update the finding status accordingly.

LDS-03 | Centralized Withdrawn Risk

Category	Severity	Location	Status
Centralization	● Centralization	XAUEAdapter.sol (05/27-b5f3b9): 275~279; XAUTStaking.sol (05/27-b5f3b9): 375~379	● Acknowledged

Description

Both `XAUEAdapter` and `XAUTStaking` implement an `emergencyWithdraw()` function that allows the `MANAGER` role to centrally withdraw assets held by the contracts.

A compromised or malicious holder of the `MANAGER` role could arbitrarily withdraw assets from the contracts, resulting in accounting inconsistencies and loss of user funds held within the protocol.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND

- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
- AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
- OR
- Remove the risky functionality.

I Alleviation

[Lista Dao, 06/03/2026]:

Acknowledged. emergencyWithdraw will only be called when emergency situations.

[CertiK, 06/04/2026]:

It is suggested to implement the aforementioned methods to avoid centralized failure. CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

We will review the transactions once contracts have been deployed and update the finding status accordingly.

LDS-04 | Excessive NAV Price Volatility Leads To Denial Of Service

Category	Severity	Location	Status
Coding Issue	● Medium	XAUEAdapter.sol (05/27-b5f3b9): 404~406	● Acknowledged

Description

The `_pushLoss` function in the XAUEAdapter contract contains a sanity check that reverts if the loss amount exceeds a configurable percentage threshold (`maxDeltaBps`) of the base value. This same check exists in the `_pushInterest` function. The purpose of this check is to defend against oracle anomalies by limiting the maximum NAV (Net Asset Value) change that can be processed in a single update. However, this mechanism creates a denial-of-service vulnerability when legitimate market conditions cause NAV price volatility beyond the configured threshold.

```
404     require(loss * 10000 <= baseValue * maxDeltaBps, "delta exceeds max");
405     IXAUTStaking(staking).decreaseTotalAssets(loss);
406 }
```

The `_pushLoss` function is called from two critical paths: `_updateVaultAssets()` for active share NAV changes and `acknowledgeReject()` for rejected redemption shares. When the NAV drops by more than `maxDeltaBps` (default 10%, maximum 30%), the transaction reverts with "delta exceeds max". This prevents the system from processing legitimate losses during periods of high market volatility. Similarly, the `_pushInterest` function would be blocked during periods of significant NAV appreciation.

Recommendation

Consider implementing a graduated approach instead of a hard revert. Options include:

1. Process only the allowed percentage of the loss/interest and defer the remainder to subsequent updates
2. Implement a circuit breaker pattern that allows emergency overrides by privileged roles

Alleviation

[Lista Dao, 06/03/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

LDS-05 | `deposit()` Does Not Round Up The Amount

Category	Severity	Location	Status
Logical Issue	Minor	XAUTStaking.sol (05/27-b5f3b9): 168~172	Resolved

Description

When calling `deposit()`, the amount paid by the user should be greater than or equal to the corresponding number of shares. Therefore, when converting shares into an amount, the result should be rounded up. However, the current implementation rounds the amount down: `XAUTStaking.sol`

```
168     if (amount > 0) {
169         shares = convertToShares(amount);
170     } else {
171 @>     amount = convertToAssets(shares);
172     }
```

`XAUTStaking.sol`

```
331     function convertToAssets(uint256 shares) public view returns (uint256) {
332         return shares.mulDiv(userTotalAssetsScaled + 1, (slisXAUE.totalSupply() + 1
) * SCALE_RATIO);
333     }
```

Recommendation

We recommend rounding the converted amount up instead.

Alleviation

[Lista Dao, 06/03/2026]: The team heeded the advice to resolve the finding and changes have been reflected in the commit [9756db1fb9c080d09add6701ef9514f646ff21cc](#).

LDS-06 Potential Bad Debt Issue When `SlisXAUE.totalSupply()` Is Zero

Category	Severity	Location	Status
Logical Issue	● Minor	XAUTStaking.sol (05/27-b5f3b9): 266~274, 284~296	● Acknowledged

Description

When all users have withdrawn their XAUT from `XAUTStaking` (including both users who have already completed `claimWithdraw()` and users who have submitted `requestWithdraw()`), there exists a period between the last withdrawal request being submitted and the final `approveRedemption` from `XAUEProtocol` to `XAUEAdapter` during which `SlisXAUE.totalSupply()` becomes zero. During this period, any gains or losses processed through `updateVaultAssets()` are not distributed to staking users in `XAUTStaking`, as the function returns immediately when the total supply is zero. Such a scenario may occur during a decline in price when users panic and withdraw all XAUT from `XAUTStaking`. `XAUTStaking.sol`

```

266 function increaseTotalAssets(uint256 amount) external {
267     require(msg.sender == adapter, "only adapter");
268     if (slisXAUE.totalSupply() == 0) {
269         emit IncreaseTotalAssetsSkipped(amount);
270         return;
271     }
272     userTotalAssetsScaled += amount * SCALE_RATIO;
273     emit IncreaseTotalAssets(amount);
274 }

```

`XAUTStaking.sol`

```

284 function decreaseTotalAssets(uint256 amount) external {
285     require(msg.sender == adapter, "only adapter");
286     if (slisXAUE.totalSupply() == 0) {
287         emit DecreaseTotalAssetsSkipped(amount);
288         return;
289     }
290     uint256 scaled = amount * SCALE_RATIO;
291     if (scaled >= userTotalAssetsScaled) {
292         scaled = userTotalAssetsScaled; // cap
293     }
294     userTotalAssetsScaled -= scaled;
295     emit DecreaseTotalAssets(amount);
296 }

```

As a result, price fluctuations during this period only affect the amount of XAUT ultimately received by `XAUEAdapter` from the withdrawal process. These gains or losses are not reflected in the accounting of `XAUTStaking` and are therefore not

reflected in the actual amount of tokens users expect to withdraw.

This may lead to a situation where the amount of XAUT ultimately received by `XAUEAdapter` is lower than the total amount of XAUT expected to be redeemed by all pending withdrawal requests in `XAUTStaking`. Consequently, some withdrawal requests may DoS.

■ Recommendation

This issue appears difficult to fully resolve under the current contract architecture. We recommend that the team ensure sufficient XAUT reserves are available to satisfy user withdrawals in such situations.

■ Alleviation

[Lista Dao, 06/03/2026]:

Acknowledged. For the final withdrawal, any loss / funding shortfall is borne by Lista DAO.

LDS-07 Withdrawal Requests May Be Blocked By The `minRedeemShares` Constraint

Category	Severity	Location	Status
Logical Issue	● Minor	XAUEAdapter.sol (05/27-b5f3b9): 191~193	● Acknowledged

Description

`xaueFundToken.requestRedemption()` enforces a minimum share requirement `minRedeemShares` :

`XAUEAdapter.sol`

```

181  function requestWithdrawFromVault(uint256 assetAmount) external onlyRole(BOT)
nonReentrant {
182      require(assetAmount > 0, "amount is zero");
183
184      _updateVaultAssets();
185
186
// shareAmount = ceil(assetAmount * 1e30 / nav). Ceil rounding ensures we redeem at
least assetAmount.

187      uint256 nav = IXAUEOracle(xaueOracle).getLatestPrice();
188      require(nav > 0, "zero nav");
189      uint256 shareAmount = assetAmount.mulDiv(1e30, nav, Math.Rounding.Ceil);
190
// Defensive pre-check: surfaces a local error instead of relying on XAUE's revert.
191  @> require(shareAmount >= IXAUEFundToken(xaueFundToken).minRedeemShares(),
"below xaue minRedeem");
192
193  @> uint256 reqId = IXAUEFundToken(xaueFundToken).requestRedemption(shareAmount
);
194
195      expectedShareBalance -= shareAmount;
196      lastVaultTotalAssets = getVaultTotalAssets();
197      emit RequestRedeemFromVault(reqId, assetAmount, shareAmount);
198  }

```

When the remaining shares held by `XAUEAdapter` fall below `minRedeemShares`, `requestWithdrawFromVault()` can no longer be executed. As a result, withdrawal requests in `XAUTStaking` will be temporarily blocked unless new funds are deposited through `XAUTStaking.deposit()`, and subsequently transferred via `depositToVault()`, causing the share balance held by `XAUEAdapter` to exceed `minRedeemShares`.

Recommendation

This issue appears difficult to fully resolve under the current architecture. The team should ensure that the remaining share balance in the contract does not fall below `minRedeemShares` .

■ Alleviation

[Lista Dao, 06/03/2026]:

Acknowledged. this is protocol design

LDS-08 | Deposit And Request Withdraw Lack On-Chain Slippage Protection

Category	Severity	Location	Status
Logical Issue, Design Issue	● Minor	XAUTStaking.sol (05/27-b5f3b9): 161, 193	● Acknowledged

Description

`XAUTStaking.deposit` and `requestWithdraw` price user actions at the exchange rate computed inside each transaction after `updateVaultAssets()`. For deposits with a fixed XAUT `amount`, users cannot require a minimum `slisXAUE` minted; for withdrawals with a fixed XAUT `amount`, users cannot cap how many shares are burned, and for withdrawals with a fixed `shares` input they cannot require a minimum XAUT amount locked in the queue. If pool state moves unfavorably between off-chain quoting and on-chain execution, transactions still succeed at worse-than-quoted terms. This is a defense-in-depth gap: partial mitigations exist via the alternate parameterization on each function (`shares`-based deposit, `amount`-based withdraw with upward share rounding), but there is no symmetric on-chain guard for the common fixed-budget paths.

Vulnerability Detail

Deposit (`amount > 0`)

When `amount > 0`, `deposit` computes minted shares solely from `convertToShares(amount)` and never checks a caller-supplied floor:

```
// File: src/slisXAUE/XAUTStaking.sol
function deposit(uint256 amount, uint256 shares, address receiver) external
whenNotPaused nonReentrant {
    require(amount > 0 || shares > 0, "amount and shares both zero");
    require(receiver != address(0), "receiver is zero");

    IXAUEAdapter(adapter).updateVaultAssets();

    if (amount > 0) {
        shares = convertToShares(amount);
    } else {
        amount = convertToAssets(shares);
    }

    require(shares > 0, "shares is zero");
    require(amount > 0, "amount is zero");
    require(amount >= minDeposit, "below min deposit");
    require(slisXAUE.totalSupply() + shares <= mintCap, "exceeds mint cap");

    slisXAUE.mint(receiver, shares);
    userTotalAssetsScaled += amount * SCALE_RATIO;
    IERC20(asset).safeTransferFrom(msg.sender, adapter, amount);

    emit Deposit(receiver, amount, shares);
}
```

The pricing function floors the share amount via `mulDiv`:

```
// File: src/slisXAUE/XAUTStaking.sol
function convertToShares(uint256 assets) public view returns (uint256) {
    return (assets * SCALE_RATIO).mulDiv(slisXAUE.totalSupply() + 1,
userTotalAssetsScaled + 1);
}
```

Deposit execution narrative

1. A user prepares a deposit of `amount` XAUT off-chain using `pricePerShare()` or `convertToShares(amount)` at block N .
2. Before the user's transaction is mined, another transaction in the same block (or the mandatory in-tx `updateVaultAssets()` after NAV movement) changes the pool rate.
3. The user's `deposit(amount, 0, receiver)` executes: NAV is synced, `shares` is recomputed, and the call succeeds with no revert bound.
4. The user pays the same `amount` but may receive fewer `slisXAUE` than quoted.

The alternate path `deposit(0, shares, receiver)` fixes the minted share count and only derives the XAUT cost (`maxAmountIn` would be the symmetric guard for that path).

Request withdraw (`requestWithdraw`)

`requestWithdraw` mirrors the deposit pattern: NAV sync, then pricing from either fixed `amount` or fixed `shares`, with no caller slippage limits:

```
// File: src/slisXAUE/XAUTStaking.sol
function requestWithdraw(uint256 amount, uint256 shares, address receiver) external
whenNotPaused nonReentrant {
    require(amount > 0 || shares > 0, "amount and shares both zero");
    require(receiver != address(0), "receiver is zero");

    IXAUEAdapter(adapter).updateVaultAssets();

    if (amount > 0) {
        shares = convertToShares(amount);
    } else {
        amount = convertToAssets(shares);
    }
    if (convertToAssets(shares) < amount) {
        shares += 1;
    }

    require(shares > 0, "shares is zero");
    require(amount > 0, "amount is zero");
    require(amount >= minWithdraw, "below min withdraw");
    require(slisXAUE.balanceOf(msg.sender) >= shares, "insufficient shares");

    slisXAUE.burn(msg.sender, shares);
    // ... queue WithdrawalRequest with locked `amount` ...
}
```

Input mode	User intent	On-chain outcome	Missing guard	Unfavorable case
<code>amount > 0</code>	Withdraw fixed XAUT	Burns computed shares (+1 wei rounding)	<code>maxShares</code>	Burns more <code>slisXAUE</code> than quoted for the same XAUT
<code>shares > 0</code>	Burn fixed shares	Locks <code>amount</code> from <code>convertToAssets</code> (floor)	<code>minAmountOut</code>	Queued XAUT less than quoted for the same shares

Recommendation

Add explicit slippage parameters to both user-facing entrypoints.

I Alleviation

[Lista Dao, 06/03/2026]:

Acknowledged.

LDS-16 | Pre-Sync On `setXAUEOracle` Blocks Oracle Migration When Current Oracle Causes Sync Revert

Category	Severity	Location	Status
Logical Issue	● Minor	XAUEAdapter.sol (06/03-92af28): 316	● Acknowledged

Description

`XAUEAdapter.setXAUEOracle` calls `_updateVaultAssets()` **before** repointing `xaueOracle`. Because `_updateVaultAssets` reads NAV from the **current** oracle and fail-closes on several anomaly conditions, any revert under the incumbent oracle prevents the MANAGER from switching to a replacement through this function. That creates a real liveness risk on the oracle-migration path precisely when migration is most needed.

Vulnerability Detail

Ordered pre-sync gates oracle replacement

```
// File: src/slisXAUE/XAUEAdapter.sol
function setXAUEOracle(address _xaueOracle) external onlyRole(MANAGER) {
    require(_xaueOracle != address(0), "xaueOracle is zero");
    _updateVaultAssets();
    xaueOracle = _xaueOracle;
    lastVaultTotalAssets = getVaultTotalAssets();
    emit SetXAUEOracle(_xaueOracle);
}
```

NatSpec (`XAUEAdapter.sol:309-314`) states the intent: settle accrued NAV under the **old** oracle, switch, then re-baseline `lastVaultTotalAssets` against the **new** oracle so a pricing-basis change is not mis-booked as interest. That ordering is sound for accounting when the pre-sync succeeds, but it makes the migration transaction **dependent on the oracle being replaced**.

The same pre-sync pattern appears on `setFeeRate` (`XAUEAdapter.sol:285-291`). `setMaxDeltaBps` is asymmetric: it pre-syncs only when **lowering** the cap and deliberately **skips** pre-sync when **raising** it, because a pending over-cap delta would revert and block the raise. `setXAUEOracle` has no analogous escape hatch.

`_updateVaultAssets` **revert surfaces that block** `setXAUEOracle`

```
// File: src/slisXAUE/XAUEAdapter.sol
function _updateVaultAssets() private {
    require(IERC20(xaueFundToken).balanceOf(address(this)) >= expectedShareBalance,
"share balance below expected");

    uint256 newVaultTotalAssets = getVaultTotalAssets();
    require(newVaultTotalAssets >= lastVaultTotalAssets, "vault value decreased");

    if (newVaultTotalAssets > lastVaultTotalAssets) {
        uint256 totalInterest = newVaultTotalAssets - lastVaultTotalAssets;
        (uint256 interestFee, uint256 netInterest) = _pushInterest(totalInterest,
lastVaultTotalAssets);
        emit UpdateVaultAssets(newVaultTotalAssets, totalInterest, interestFee,
netInterest);
    }
    lastVaultTotalAssets = newVaultTotalAssets;
}
```

`getVaultTotalAssets()` prices shares through the **current** `xaueOracle` :

```
// File: src/slisXAUE/XAUEAdapter.sol
function getVaultTotalAssets() public view returns (uint256) {
    uint256 nav = IXAUEOracle(xaueOracle).getLatestPrice();
    return expectedShareBalance.mulDiv(nav, 1e30);
}
```

Under the incumbent oracle, `setXAUEOracle` reverts when **any** of the following hold at call time:

Condition	Revert	Migration blocked?
<code>balanceOf(xaueFundToken) < expectedShareBalance</code>	"share balance below expected"	Yes
Oracle-implied <code>newVaultTotalAssets < lastVaultTotalAssets</code>	"vault value decreased"	Yes — cap-raise does not help
Pending gain exceeds <code>maxDeltaBps</code> (default 10%)	"delta exceeds max" via <code>_pushInterest</code>	Yes, until backlog cleared under a higher cap or separate successful sync

The second row is the primary oracle-migration failure mode: if the **old** oracle reads below the adapter's stored baseline (misconfiguration, decommissioned feed returning zero/stale low, cross-environment mismatch, or any non-monotonic

reading relative to `lastVaultTotalAssets`), `_updateVaultAssets` fail-closes and `setXAUEOracle` cannot complete—even though the MANAGER's goal is to **replace** that oracle with a healthy one.

The third row mirrors the cap-backlog scenario: a large positive delta accrued since the last sync (e.g. long heartbeat outage) causes `"delta exceeds max"` . Unlike `setMaxDeltaBps` when raising the cap, `setXAUEOracle` always attempts the pre-sync first, so the MANAGER must clear the backlog through a separate path before migration can proceed.

■ Recommendation

Consider adding fail-open migration path in the `setXAUEOracle` function.

■ Alleviation

[Lista Dao, 06/08/2026]:

Acknowledged.

LDS-09 | Variable And Comment Inconsistency

Category	Severity	Location	Status
Inconsistency	● Informational	XAUTStaking.sol (05/27-b5f3b9): 68~69	● Resolved

Description

The comments indicate that `mintCap` is not intended to be visible to users.

`XAUTStaking.sol`

```
68
/// @notice Maximum total shares (18-dec). Risk control; users do not see this
number.

69     uint256 public mintCap;
```

However, all data stored on the blockchain is publicly accessible. We would like to confirm whether this behavior is consistent with the intended design.

Recommendation

We would like to confirm whether this behavior is consistent with the intended design.

Alleviation

[Lista Dao, 06/03/2026]:

Fixed commit: 9756db1fb9c080d09add6701ef9514f646ff21cc

LDS-10 | User-Facing Transfers Credit Nominal Amount Instead Of Received Balance

Category	Severity	Location	Status
Volatile Code	● Informational	XAUTStaking.sol (05/27-b5f3b9): 180~181, 306~307	● Acknowledged

Description

`XAUTStaking` records economic exposure using the caller-supplied `amount` parameter rather than the actual token balance received by the destination contract. In `deposit`, `userTotalAssetsScaled` is increased by the nominal deposit before `safeTransferFrom` moves tokens to the adapter. In `finishWithdraw`, `withdrawQuota` is increased by the nominal redemption amount after `safeTransferFrom` from the adapter. If the configured `asset` is a fee-on-transfer or otherwise deflationary ERC-20, the protocol can over-account liabilities or withdrawal funding relative to tokens actually held, leading to failed claims or diluted backing for remaining users. Under a standard non-fee XAUT implementation this is primarily a defense-in-depth and asset-compatibility gap.

Vulnerability Detail

`finishWithdraw` — inflated `withdrawQuota`

Only the adapter may call `finishWithdraw`. When `amount > 0`, the contract pulls tokens and credits `withdrawQuota` by the requested gross value:

```
// File: src/slisXAUE/XAUTStaking.sol
function finishWithdraw(uint256 amount) external {
    require(msg.sender == adapter, "only adapter");

    if (amount > 0) {
        IERC20(asset).safeTransferFrom(msg.sender, address(this), amount);
        withdrawQuota += amount;
    }

    for (uint256 i = confirmedBatchId + 1; i <= currentBatchId; i++) {
        uint256 batchAmount = totalWithdrawAmountInBatch[i];
        if (batchAmount > withdrawQuota) break;
        confirmedBatchId = i;
        withdrawQuota -= batchAmount;
        emit FinishWithdraw(i, batchAmount);
    }
}
```

The FIFO loop treats `withdrawQuota` as funds available to confirm batches. It never compares `withdrawQuota` or

cumulative confirmed liabilities against `IERC20(asset).balanceOf(address(this))`.

`deposit` — inflated pool liabilities

`deposit` applies the same pattern on the user entry path:

```
// File: src/slisXAUE/XAUTStaking.sol
function deposit(uint256 amount, uint256 shares, address receiver) external
whenNotPaused nonReentrant {
    // ... pricing after updateVaultAssets() ...

    slisXAUE.mint(receiver, shares);
    userTotalAssetsScaled += amount * SCALE_RATIO;
    IERC20(asset).safeTransferFrom(msg.sender, adapter, amount);

    emit Deposit(receiver, amount, shares);
}
```

`userTotalAssetsScaled` and minted `shares` are derived from the nominal `amount`, not from tokens the adapter actually receives. A fee-on-transfer asset therefore mints claims on the full quoted deposit while the adapter vault holds less XAUT, mispricing `convertToShares` / `convertToAssets` for other holders.

Relationship between the two paths

Function	Nominal field credited	Token destination	Downstream risk
<code>deposit</code>	<code>userTotalAssetsScaled</code>	<code>adapter</code>	Over-minted / over-priced shares vs real backing
<code>finishWithdraw</code>	<code>withdrawQuota</code>	<code>staking (this)</code>	Over-confirmed batches vs claimable balance

Neither path uses `balanceBefore` / `balanceAfter` on the relevant receiver.

Recommendation

Measure received tokens everywhere economics are updated or explicitly disallow fee-on-transfer / rebasing tokens.

Alleviation

[Lista Dao, 06/03/2026]:

Acknowledged. asset is fixed to XAUE

LDS-11 | DecreaseTotalAssets Event Emits Uncapped Input Amount

Category	Severity	Location	Status
Logical Issue	● Informational	XAUTStaking.sol (05/27-b5f3b9): 295	● Resolved

Description

`XAUTStaking.decreaseTotalAssets` caps the storage reduction of `userTotalAssetsScaled` to the current ledger balance but emits `DecreaseTotalAssets(amount)` using the **uncapped** adapter argument. Off-chain indexers, subgraphs, and dashboards that reconstruct staking TVL or loss history from events can therefore over-subtract losses or record **phantom** decreases when the cap binds or when `userTotalAssetsScaled` is already zero while `slisXAUE.totalSupply() > 0`.

```
// File: src/slisaue/XAUTStaking.sol
function decreaseTotalAssets(uint256 amount) external {
    require(msg.sender == adapter, "only adapter");
    if (slisXAUE.totalSupply() == 0) {
        emit DecreaseTotalAssetsSkipped(amount);
        return;
    }
    uint256 scaled = amount * SCALE_RATIO;
    if (scaled >= userTotalAssetsScaled) {
        scaled = userTotalAssetsScaled; // cap
    }
    userTotalAssetsScaled -= scaled;
    emit DecreaseTotalAssets(amount);
}
```

`SCALE_RATIO` is `1e12` (6-decimal XAUT → 18-decimal scaled assets). NatSpec documents that excess loss is **silently dropped** when the adapter over-reports relative to remaining user backing. The applied delta is `scaled` (after cap), but the event always logs the requested `amount` in 6-decimal XAUT units.

Recommendation

Emit the executed amount after capping.

Alleviation

[Lista Dao, 06/03/2026]: Fixed commit: 9756db1fb9c080d09add6701ef9514f646ff21cc

[CertiK, 06/04/2026]: The team have removed `decreaseTotalAssets` in the fixed version and changes have been reflected in the commit [9756db1fb9c080d09add6701ef9514f646ff21cc](#).

LDS-13 | Fee Collection During NAV Price Fluctuations

Category	Severity	Location	Status
Design Issue	● Informational	XAUEAdapter.sol (05/27-b5f3b9): 389–398	● Acknowledged

Description

`XAUEAdapter._updateVaultAssets` treats every NAV increase relative to `lastVaultTotalAssets` (the last sync baseline) as fresh profit via `_pushInterest`, charging `feeRate` and pushing only the net amount to `XAUTStaking`. This is **not** a High-Water Mark (HWM) performance-fee model: there is no separate peak NAV tracker, and recovery gains that merely restore a prior synced loss are still fee-bearing. Combined with the explicit policy that `_pushLoss` does **not** claw back fees already credited on earlier upside syncs, repeating down-then-up recognition cycles leaks value from holders to the protocol fee ledger on each recovery leg.

In traditional fund / hedge-fund performance fees, the **High-Water Mark** is the highest NAV (or net asset value per share) at which a performance fee was last charged—or, equivalently, the peak from which “new profit” is measured.

Typical HWM lifecycle:

Event	NAV	HWM	Feeable gain	Fee charged?
Start	100	100	—	—
Rise to 110, fee event	110	110 (raised)	10	Yes (on 10)
Fall to 95, loss recognized	95	110 (unchanged)	—	No
Recover to 110	110	110 (unchanged)	0 (only restored to HWM)	No
Rise to 115	115	115 (raised)	5 (above HWM)	Yes (on 5)

Current `XAUEAdapter` behavior uses `lastVaultTotalAssets` as the sole comparison point for both gain and loss legs. That variable is rebased to `newVaultTotalAssets` on **every** successful sync—up or down—so it tracks the **last accounting snapshot**, not the **historical NAV peak for fee purposes**:

Event	NAV	<code>lastVaultTotalAssets</code> after sync	Feeable gain (code)	Fee charged?
Start	100	100	—	—

Event	NAV	lastVaultTo talAssets after sync	Feeable gain (code)	Fee charged?
Rise to 110, sync	110	110	10	Yes
Fall to 95, sync	95	95 (lowered)	—	No (loss to holders)
Recover to 110, sync	110	110	10 (= 110 – 95)	Yes (treated as new profit)
Fall to 100, sync	100	100	—	No; prior fee on recovery leg not clawed back

This creates a potential asymmetry where:

- 1. Protocol benefits from volatility:** The protocol collects fees on upward movements but doesn't refund fees during downward movements
- 2. Users bear volatility risk:** Users experience the full impact of NAV declines but only receive net gains after fees
- 3. Compounding effect:** In highly volatile markets, frequent fee collection could significantly reduce user returns

```

389     require(gain * 10000 <= baseValue * maxDeltaBps, "delta exceeds max");
390     gainFee = gain.mulDiv(feeRate, PRECISION);
391     fee += gainFee;
392     netGain = gain - gainFee;
393     if (netGain > 0) {
394         IXAUTStaking(staking).increaseTotalAssets(netGain);
395     }
396 }
397
398 /**

```

This design raises an important discussion point about the fairness and economic implications of fee collection during volatile market conditions.

Recommendation

We would like to confirm with the team whether the current implementation is intended by design.

Alleviation

[Lista Dao, 06/03/2026]:

Acknowledged. It is protocol design.

LDS-14 Confirmation Of `depositToVault` , `requestWithdrawFromVault` And `acknowledgeReject` Delays In The Protocol

Category	Severity	Location	Status
Design Issue	● Informational	XAUEAdapter.sol (05/27-b5f3b9): 159~171, 181~198, 308~348	● Acknowledged

Description

Regarding several issues arising from the protocol's time-delay design, we would like to seek clarification from the team:

1. When a user calls `deposit()` in `XAUTStaking` , the funds are not immediately deposited into the XAUE protocol. Instead, they are transferred into the XAUE protocol only after a BOT subsequently calls `XAUEAdapter.depositToVault()` . As a result, the XAUT token flow is `XAUTStaking -> XAUEAdapter -> XAUEProtocol` . During this intermediate period, the user's funds do not immediately affect the protocol, effectively reducing the impact of token price fluctuations on all `SLISXAUE` holders during that time. We would like to confirm whether this behavior is an intended design choice.
2. When a user calls `requestWithdraw()` in `XAUTStaking` , the funds are not immediately transferred from the XAUE protocol to the user. The withdrawal process involves multiple steps: `XAUTStaking.requestWithdraw()` -> `XAUEAdapter.requestWithdrawFromVault()` -> `XAUEProtocol.approveRedemption()` -> `XAUEAdapter.finishEarnPoolWithdraw()` -> `XAUTStaking.claimWithdraw()` . During the period between `XAUTStaking.requestWithdraw()` and `XAUEProtocol.approveRedemption()` , the corresponding XAUT remains in `XAUEProtocol` . As a result, any price fluctuations during this period continue to affect the remaining `SLISXAUE` holders, effectively amplifying the impact of token price fluctuations on all `SLISXAUE` . At the same time, the assets pending withdrawal continue to contribute to the gains or losses borne by holders who have not initiated withdrawals. We would like to confirm whether this behavior is an intended design choice.
3. Similarly, in the case of a failed withdrawal that is eventually handled through `acknowledgeReject()` , price fluctuations occurring between the withdrawal request submission and the execution of `acknowledgeReject()` continue to affect the XAUT value corresponding to XAUE. During this period, the assets pending withdrawal still contribute to the gains or losses borne by holders who have not initiated withdrawals. We would like to confirm whether this behavior is an intended design choice.

Recommendation

We would like to confirm whether this behavior is an intended design choice.

Alleviation

[Lista Dao, 06/03/2026]: Acknowledged. all three are intended async-bridge design

LDS-15 | Queued Withdrawal Amount Locking Leads To Loss Shifting Onto Remaining Shareholders

Category	Severity	Location	Status
Coding Issue, Design Issue	● Informational	XAUEAdapter.sol (05/27-b5f3b9): 181, 362; XAUTStaking.sol (05/27-b5f3b9): 193	● Acknowledged

Description

The `requestWithdraw` function is intended to let a user exit the strategy by burning `shares`, fixing a withdrawal `amount`, and moving the request into a queued batch that will be funded later. However, once the request is created, the contract immediately subtracts `amount` from `userTotalAssetsScaled`, which removes the queued portion from the active share accounting before the underlying assets are actually redeemed. After that, `requestWithdrawFromVault` also reduces `expectedShareBalance` and resets `lastVaultTotalAssets` to the smaller active slice, so any later NAV decrease processed by `_updateVaultAssets` is applied only through `decreaseTotalAssets` to the remaining active holders. As a result, users who already entered the withdrawal queue can still claim their previously locked `amount`, while the full subsequent loss is concentrated on holders who stayed in the pool, creating a value transfer from active shareholders to queued withdrawers.

Recommendation

Do not fully remove queued withdrawals from loss sharing until the underlying assets backing those requests are actually segregated or settled. One approach is to keep queued requests represented by a dedicated liability/share bucket that continues to participate pro rata in gains and losses until `finishWithdraw` confirms funding.

Alleviation

[Lista Dao, 06/03/2026]:

The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

OPTIMIZATIONS | LISTA DAO - SLISXAUE

ID	Title	Category	Severity	Status
LDS-12	Unbounded Withdrawal Request Arrays Lead To Degrading Bulk Read Scalability	Code Optimization	Optimization	● Resolved

LDS-12 | Unbounded Withdrawal Request Arrays Lead To Degrading Bulk Read Scalability

Category	Severity	Location	Status
Code Optimization	● Optimization	XAUTStaking.sol (05/27-b5f3b9): 226~227, 345~347	● Resolved

Description

The `requestWithdraw` function appends each new withdrawal request into `userWithdrawalRequests[receiver]`, and `getUserWithdrawalRequests` is intended to expose the full pending request list for a receiver in one call. The issue is that there is no per-user or global bound on how large that array can grow, so the cost of returning the full array increases monotonically as more requests accumulate. This does not by itself block core withdrawal settlement on-chain, but it creates a real scalability problem for frontends, indexers, RPC providers, or other consumers that rely on the bulk getter to inspect pending requests. The risk becomes easier to realize if `minWithdraw` is configured very low by a privileged `MANAGER`, because smaller requests reduce the capital needed to grow a receiver's queue aggressively.

Recommendation

Prefer a paginated retrieval pattern instead of returning the full array in one view. For example, store requests with explicit ids and expose `getUserWithdrawalRequestCount` plus `getUserWithdrawalRequests(user, start, end)`, or maintain a queue head/tail model that allows bounded reads.

Alleviation

[Lista Dao, 06/03/2026]:

The team heeded the advice and resolved the issue by using the paginated retrieval pattern in commit [9756db1fb9c080d09add6701ef9514f646ff21cc](#)

APPENDIX | LISTA DAO - SLISXAUE

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Code Optimization	No description available.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Lista Dao - slisXAUE Security Assessment | CertiK Assessed on Jun 8th, 2026 | Copyright © CertiK

